

TECHNICAL PROJECT DOCUMENTATION

RetailMart Data Pipeline Project

1. Project Overview

RetailMart Pvt. Ltd. collects daily sales data from multiple retail stores. The data is stored in different CSV files and contains real-world data quality issues such as missing values, duplicate records, and incorrect data formats.

The objective of this project is to build a simple data pipeline that ingests raw data, performs data cleaning and transformation, stores the processed data into a database, and generates business insights for reporting.

The pipeline combines sales transactions with product and store information to create a final analytical dataset that helps the business understand:

- Best-selling products across different cities
- Revenue generated by each store per day
- Data quality issues in products and stores
- Overall sales performance insights

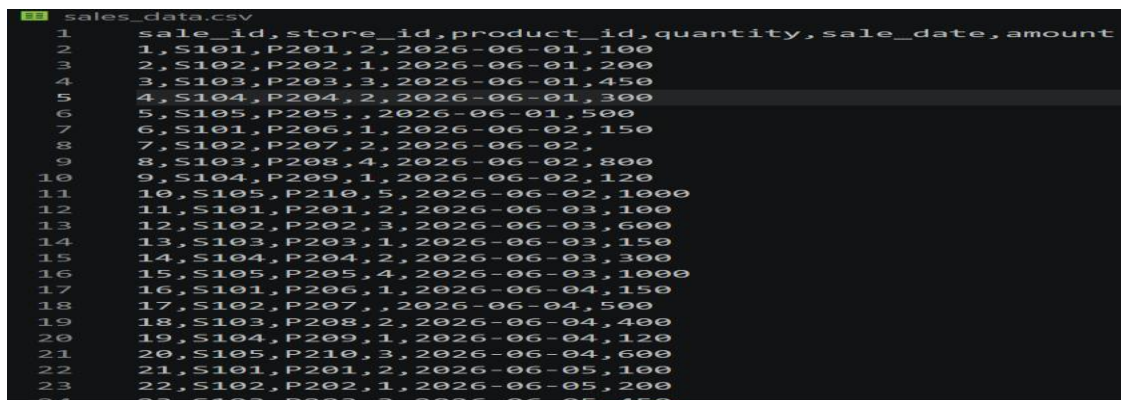
2. Project Approach

The project follows an ETL (Extract, Transform, Load) approach.

Step 1: Data Ingestion (Extract)

The first step is to collect data from the available CSV files:

- sales_data.csv



```
sales_data.csv
1,sale_id,store_id,product_id,quantity,sale_date,amount
2,1,S101,P201,2,2026-06-01,100
3,2,S102,P202,1,2026-06-01,200
4,3,S103,P203,3,2026-06-01,450
5,4,S104,P204,2,2026-06-01,300
6,5,S105,P205,,2026-06-01,500
7,6,S101,P206,1,2026-06-02,150
8,7,S102,P207,2,2026-06-02,
9,8,S103,P208,4,2026-06-02,800
10,9,S104,P209,1,2026-06-02,120
11,10,S105,P210,5,2026-06-02,1000
12,11,S101,P201,2,2026-06-03,100
13,12,S102,P202,3,2026-06-03,600
14,13,S103,P203,1,2026-06-03,150
15,14,S104,P204,2,2026-06-03,300
16,15,S105,P205,4,2026-06-03,1000
17,16,S101,P206,1,2026-06-04,150
18,17,S102,P207,,2026-06-04,500
19,18,S103,P208,2,2026-06-04,400
20,19,S104,P209,1,2026-06-04,120
21,20,S105,P210,3,2026-06-04,600
22,21,S101,P201,2,2026-06-05,100
23,22,S102,P202,1,2026-06-05,200
24,23,S103,P203,3,2026-06-05,450
```

- products.csv

```

products.csv
1  product_id,product_name,category,price
2  P201,Soap,FMCG,50
3  P202,Shampoo,FMCG,200
4  P203,Toothpaste,FMCG,150
5  P204,Biscuit,Food,150
6  P205,Coffee,Food,250
7  P206,Tea,Food,150
8  P207,Juice,Beverage,250
9  P208,Milk,Dairy,200
10 P209,Butter,Dairy,120
11 P210,Cheese,Dairy,200
12 P211,Rice,Grocery,500
13 P212,Wheat,Grocery,400
14 P213,Sugar,Grocery,50
15 P214,Salt,Grocery,20
16 P215,Oil,Grocery,180
17 P216,Pen,Stationery,10
18 P217,Notebook,Stationery,40
19 P218,Pencil,Stationery,5
20 P219,Eraser,Stationery,3
21 P220,Marker,Stationery,25
22 P221,Chips,Snacks,30
23 P222,Cookies,Snacks,60
24 P223,Chocolate,Snacks,80

```

- stores.csv

```

stores.csv
4  S103,Mumbai Hub,Mumbai,West
5  S104,Pune Center,Pune,West
6  S105,Bangalore Mart,Bangalore,South
7  S106,Chennai Point,Chennai,South
8  S107,Hyderabad Store,Hyderabad,South
9  S108,Kolkata Market,Kolkata,East
10 S109,Ahmedabad Shop,Ahmedabad,West
11 S110,Lucknow Retail,Lucknow,North
12 S111,Patna Store,Patna,East
13 S112,Bhopal Mart,Bhopal,Central
14 S113,Indore Center,Indore,Central
15 S114,Surat Point,Surat,West
16 S115,Nagpur Retail,Nagpur,Central
17 S116,Kanpur Shop,Kanpur,North
18 S117,Agra Store,Agra,North
19 S118,Noida Mart,Noida,North
20 S119,Gurgaon Plaza,Gurgaon,North
21 S120,Coimbatore Hub,Coimbatore,South
22 S121,Kochi Center,Kochi,South
23 S122,Vizag Point,Vizag,South
24 S123,Ranchi Retail,Ranchi,East
25 S124,Guwahati Shop,Guwahati,East
26 S125,Bhubaneswar Store,Bhubaneswar,East
27

```

Using Python Pandas, all three files are loaded into separate DataFrames.

Initial data inspection is performed by:

- Checking dataset shape
- Displaying first five records
- Identifying missing values in each column

This helps understand the structure and quality of raw data before processing.

Step 2: Data Cleaning (Transform)

The raw sales data contains common data issues, so cleaning operations are performed.

The following cleaning steps are applied:

Removing Duplicate Records

Duplicate transactions are identified and removed from the sales dataset to avoid incorrect revenue calculations.

Handling Missing Values

- Missing values in the quantity column are replaced with 0.
- Rows where the amount field is missing are removed because revenue calculation cannot be performed without transaction amounts.

Data Type Conversion

Data formats are standardized:

- sale_date is converted into datetime format.
- amount is converted into float data type.

These steps ensure the data is accurate and ready for analysis.

Step 3: Data Transformation

After cleaning, the datasets are combined using joins.

The sales data is merged with:

- Products dataset using product_id
- Stores dataset using store_id

A final merged dataset is created containing:

- Transaction details
- Product information
- Store details

A new column called total_revenue is created using:

$\text{total_revenue} = \text{quantity} \times \text{price}$

Using NumPy, statistical analysis is performed to calculate:

- Average revenue
- Maximum revenue
- Minimum revenue

Further analysis is performed by grouping data based on city to identify total revenue generated from each location.

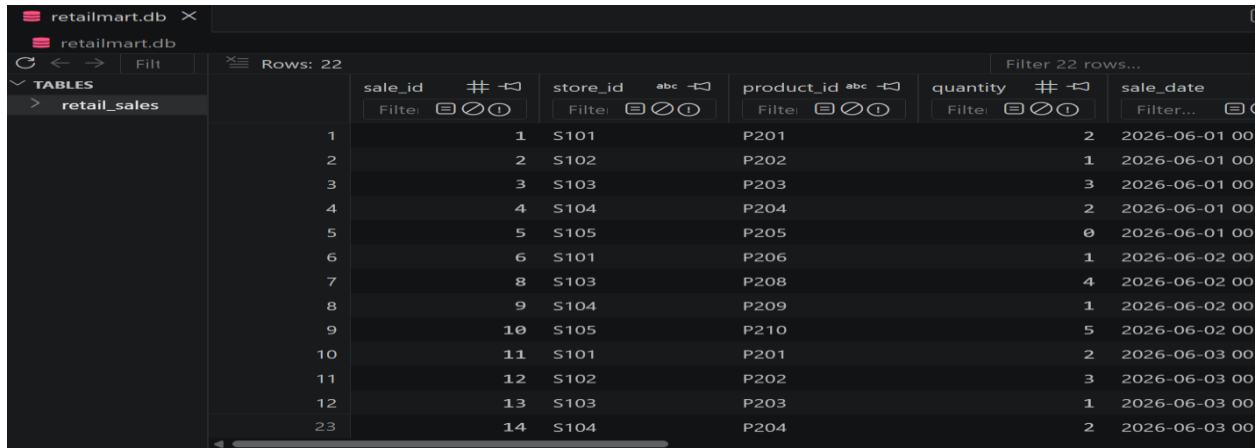
Step 4: Data Loading

The processed dataset is loaded into a SQLite database.

A database table named:

retail_sales

is created to store the cleaned and transformed data.



The screenshot shows a SQLite database viewer interface. The top bar indicates the database is 'retailmart.db'. Below the toolbar, the 'TABLES' section shows 'retail_sales' selected. The main area displays 22 rows of data. The columns are: sale_id, store_id, product_id, quantity, and sale_date. The data is as follows:

sale_id	store_id	product_id	quantity	sale_date
1	S101	P201	2	2026-06-01 00
2	S102	P202	1	2026-06-01 00
3	S103	P203	3	2026-06-01 00
4	S104	P204	2	2026-06-01 00
5	S105	P205	0	2026-06-01 00
6	S101	P206	1	2026-06-02 00
7	S103	P208	4	2026-06-02 00
8	S104	P209	1	2026-06-02 00
9	S105	P210	5	2026-06-02 00
10	S101	P201	2	2026-06-03 00
11	S102	P202	3	2026-06-03 00
12	S103	P203	1	2026-06-03 00
13	S104	P204	2	2026-06-03 00

SQL queries are executed to generate business reports such as:

- Top 3 best-selling products by quantity sold
- Total revenue generated per store per day

Step 5: Reporting and Insights

A summary report is generated using Python.

The report provides:

- Total number of transactions
- Total revenue generated
- Top performing city
- Best-selling product

These insights help the business team make better decisions using data-driven analysis.

Step 6: Pipeline Automation and Error Handling

A function named run_pipeline() is created to execute the complete workflow in one call.

The pipeline includes:

1. Data loading
2. Data cleaning
3. Data transformation
4. Database loading

Basic exception handling is implemented using try-except blocks.

If any input file is missing or an unexpected error occurs, the pipeline displays an appropriate error message instead of stopping unexpectedly.

3. Technology Stack Used

Python

Python is used as the main programming language because it provides powerful libraries for data processing, automation, and database integration.

It is widely used in data engineering workflows because of its simplicity and flexibility.

Pandas

Pandas is used for handling CSV files and performing data manipulation.

It helps in:

- Loading datasets
- Cleaning missing values
- Removing duplicates
- Merging multiple datasets
- Performing grouping and aggregation operations

Pandas is one of the most commonly used libraries for data analysis and ETL pipelines.

NumPy

NumPy is used for numerical calculations and statistical operations.

In this project, it is used to calculate:

- Mean revenue
- Maximum revenue
- Minimum revenue

It provides fast and efficient mathematical operations on datasets.

SQLite

SQLite is used as the database for storing the final processed data.

It is selected because:

- It is lightweight and easy to integrate with Python
- It does not require a separate database server
- It supports SQL queries for data analysis

The cleaned dataset is stored in the retail_sales table for reporting purposes.

SQL

SQL is used for querying the stored data.

SQL helps generate business reports such as:

- Top-selling products
- Store-wise daily revenue

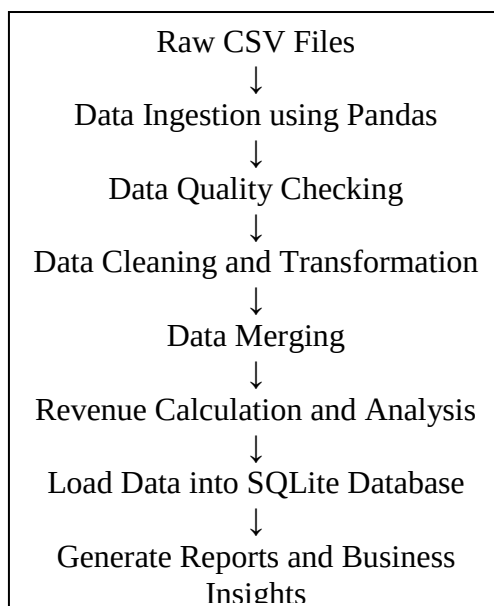
It provides an efficient way to analyze structured data stored in databases.

CSV Files

CSV files are used as input data sources because they are commonly used for exchanging and storing tabular business data.

The project simulates a real-world situation where organizations receive raw data from multiple sources in CSV format.

4. Project Workflow



Code:-

```
import pandas as pd

import numpy as np

import sqlite3


def run_pipeline():

    try:

        # Load files

        sales_df = pd.read_csv("sales_data.csv")

        products_df = pd.read_csv("products.csv")

        stores_df = pd.read_csv("stores.csv")


        print("Files loaded successfully!")


        # Remove duplicates

        sales_df = sales_df.drop_duplicates()


        # Handle missing values

        sales_df['quantity'] = sales_df['quantity'].fillna(0)

        sales_df = sales_df.dropna(subset=['amount'])


        # Convert datatypes

        sales_df['sale_date'] = pd.to_datetime(sales_df['sale_date'])
```

```
sales_df['amount'] = sales_df['amount'].astype(float)
```

```
# Merge data
```

```
merged_df = pd.merge(
```

```
    sales_df,
```

```
    products_df,
```

```
    on='product_id',
```

```
    how='left'
```

```
)
```

```
merged_df = pd.merge(
```

```
    merged_df,
```

```
    stores_df,
```

```
    on='store_id',
```

```
    how='left'
```

```
)
```

```
# Create revenue column
```

```
merged_df['total_revenue'] = (
```

```
    merged_df['quantity'] * merged_df['price']
```

```
)
```

```
# Revenue by city
```

```
city_revenue = (  
    merged_df.groupby('city')['total_revenue']  
    .sum()  
    .sort_values(ascending=False)  
)  
  
# Load into SQLite  
conn = sqlite3.connect("retailmart.db")  
  
merged_df.to_sql(  
    "retail_sales",  
    conn,  
    if_exists="replace",  
    index=False  
)  
  
conn.close()  
  
# Summary Report  
total_transactions = len(merged_df)  
total_revenue = merged_df['total_revenue'].sum()  
  
top_city = city_revenue.idxmax()
```

```

top_product = (

    merged_df.groupby('product_name')['quantity']

    .sum()

    .idxmax()

)

print("\n===== SUMMARY REPORT =====")

print("Total Transactions :", total_transactions)

print("Total Revenue      :", total_revenue)

print("Top Selling City   :", top_city)

print("Top Selling Product:", top_product)

print("\nPipeline executed successfully!")

except FileNotFoundError as e:

    print("File not found:", e)

except Exception as e:

    print("An error occurred:", e)

```

Output:-

```

PS C:\Users\DELL\Desktop\technical assessment> python retail_pipeline.py
Files loaded successfully!
Sales Data Shape: (25, 6)
  sale_id  store_id  product_id  quantity  sale_date  amount
0         1     S101         P201         2.0  2026-06-01    100.0
1         2     S102         P202         1.0  2026-06-01    200.0
2         3     S103         P203         3.0  2026-06-01    450.0
3         4     S104         P204         2.0  2026-06-01    300.0
4         5     S105         P205         NaN  2026-06-01    500.0

Products Data Shape: (25, 4)
  product_id  product_name  category  price
0         P201         Soap      FMCG      50
1         P202      Shampoo      FMCG     200
2         P203      Toothpaste  FMCG     150
3         P204         Biscuit    Food     150
4         P205         Coffee     Food     250

Stores Data Shape: (25, 4)
  store_id  store_name  city  region
0     S101  Jaipur Central  Jaipur  North
1     S102   Delhi Plaza   Delhi  North
2     S103   Mumbai Hub   Mumbai  West
3     S104   Pune Center    Pune  West
4     S105  Bangalore Mart  Bangalore  South

```

```

Sales Data Missing Values:
sale_id      0
store_id     0
product_id   0
quantity     2
sale_date    0
amount       1
dtype: int64

Products Data Missing Values:
product_id    0
product_name  0
category      0
price         0
dtype: int64

Stores Data Missing Values:
store_id      0
store_name    0
city          0
region        0
dtype: int64
Number of duplicate rows: 2
Shape after removing duplicates: (23, 6)
Shape after cleaning: (22, 6)
Merged Data Shape: (22, 12)

```

```

Merged Data Shape: (22, 12)
  sale_id store_id product_id quantity  ... price  store_name  city region
0        1      S101      P201        2.0  ...   50  Jaipur Central  Jaipur  North
1        2      S102      P202        1.0  ...  200  Delhi Plaza    Delhi  North
2        3      S103      P203        3.0  ...  150  Mumbai Hub    Mumbai  West
3        4      S104      P204        2.0  ...  150  Pune Center    Pune    West
4        5      S105      P205        0.0  ...  250  Bangalore Mart  Bangalore  South

[5 rows x 12 columns]
Mean Revenue: 331.3636363636364
Maximum Revenue: 1000.0
Minimum Revenue: 0.0
Total revenue generated per city
city
Bangalore    2600.0
Mumbai       2250.0
Delhi        1000.0
Pune         840.0
Jaipur       600.0
Name: total_revenue, dtype: float64
Rows in retail_sales table: 22
('Cheese', 8.0)
('Toothpaste', 7.0)
('Soap', 6.0)
('Mumbai Hub', '2026-06-01 00:00:00', 450.0)
('Pune Center', '2026-06-01 00:00:00', 300.0)
('Delhi Plaza', '2026-06-01 00:00:00', 200.0)

```

```

===== SUMMARY REPORT =====
Total Transactions : 22
Total Revenue      : 7290.0
Top Selling City   : Bangalore
Top Selling Product: Cheese

Pipeline executed successfully!

```

5. Conclusion

This project demonstrates a complete beginner-level data engineering pipeline using Python. It performs data ingestion, cleaning, transformation, database loading, and reporting.

The solution helps RetailMart convert raw sales data into meaningful business information that can support decision-making and future analytics requirements.